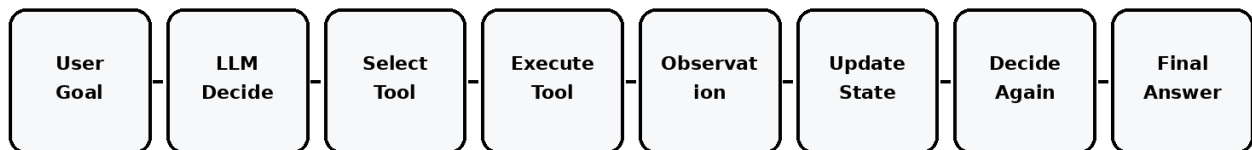


How an Ordinary LLM Becomes an AI Agent

LLM → Decision Loop → Tool Selection → Tool Execution → Observation → Memory/State → Next Action → Final Answer

Core AI Agent Loop



1. From LLM to AI Agent

An ordinary LLM primarily receives context and generates output tokens. An AI agent adds a control and decision loop around the model. This allows the system to determine that text generation alone is insufficient, select an external action, execute a tool, inspect the result, update state, and decide what to do next.

2. A Simple Example

Suppose the user asks: “Find today's weather and tell me whether I should carry an umbrella.” A plain LLM does not inherently possess live weather data. An agent can decide that current information is required, invoke a weather service, receive an observation, and then formulate a recommendation.

3. User Goal

Agentic tasks are often expressed as goals rather than simple questions. For example: “Find the cheapest flight from Bengaluru to Delhi next Friday and email me the best three options.” This requires a sequence of actions:

interpret the request, obtain current flight information, compare results, select candidates, compose a summary, and potentially send an email.

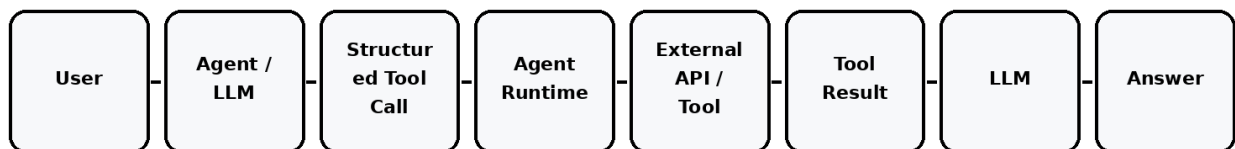
4. LLM as Decision Engine

The LLM often serves as the decision component. It can receive system instructions, the user's goal, available tool descriptions, conversation context, current task state, and previous tool observations. It then determines whether to answer directly or request an action.

5. Tool Selection

Tools extend the system beyond text generation. Common tools include web search, calculators, weather services, databases, email, calendars, Python/code execution, CRM/ERP systems, file retrieval, and other APIs.

Tool-Augmented LLM



6. Structured Tool Calls

Instead of merely saying that it will use a tool, the model can produce a structured tool request that the agent runtime can validate and execute. Conceptually:

```
{
  "tool": "weather",
  "location": "Bengaluru",
  "date": "today"
}
```

7. Tool Execution and Observation

The external action is generally executed by an agent runtime or orchestration layer rather than by the language model itself. The tool result is returned to the system and becomes an observation. The LLM can then use that observation as additional context for its next decision.

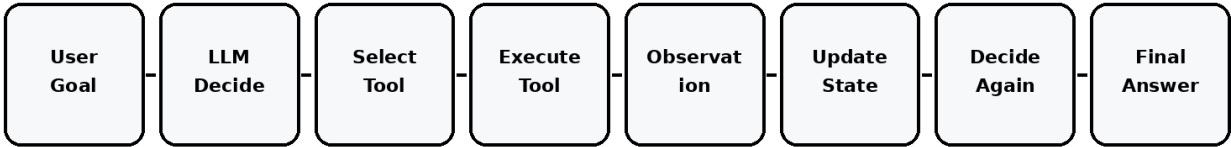
8. Memory and State

Agents often need information to persist across multiple steps. Working state may contain the current goal, completed actions, intermediate results, constraints, and pending steps. Memory can also refer to conversation information, long-term persisted information, or external stores such as databases, files, or vector indexes.

Type	Purpose
Working state	Information needed during the current task
Conversation context	Relevant information from the ongoing dialogue
Long-term memory	Persisted information useful across interactions
External memory	Databases, vector stores, files and knowledge bases

9. The Agent Control Loop

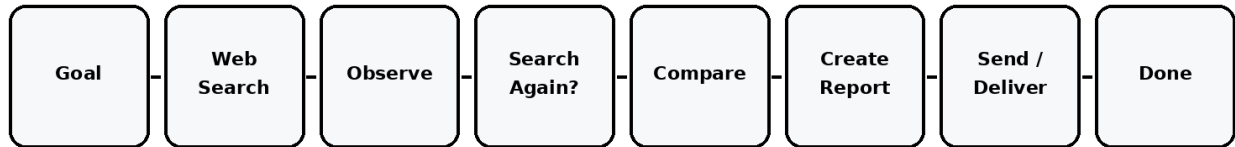
Core AI Agent Loop



The central execution cycle can be summarized as: Goal → Decide → Act → Observe → Update State → Decide Again → Finish. A single task may involve several iterations and several different tools.

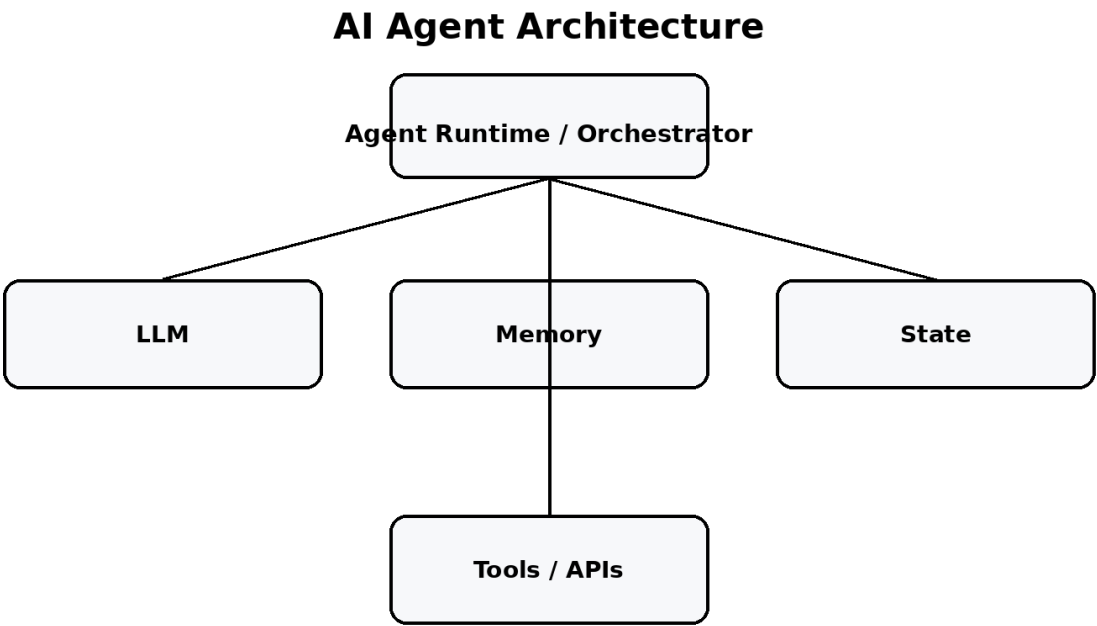
10. Example: Research Agent

Example: Research Agent Workflow



For a task such as researching cloud providers, comparing services, generating a report, and delivering it, the LLM coordinates specialized capabilities. Search retrieves information; code can perform calculations; a document component can create the report; and an email or collaboration tool can deliver it.

11. Agent Runtime / Orchestrator



The orchestrator coordinates model calls, tool execution, state, retries, limits and stopping conditions.

A production agent normally contains more than an LLM. The runtime or orchestrator can manage tool definitions, model calls, tool execution, state, retries, errors, permissions, logging, tracing, token/cost limits, and stopping conditions.

12. Stopping Conditions

An agent must know when to stop. Typical conditions include successful goal completion, maximum step count, time or cost limits, repeated errors, insufficient permissions, or a requirement for human approval. These controls reduce the risk of uncontrolled loops.

13. Human-in-the-Loop

High-impact actions can require explicit human approval before execution. Examples include purchases, financial transactions, deleting data, changing permissions, or sending consequential external communications. The agent can prepare the action and pause at an approval checkpoint.

14. LLM vs RAG vs AI Agent

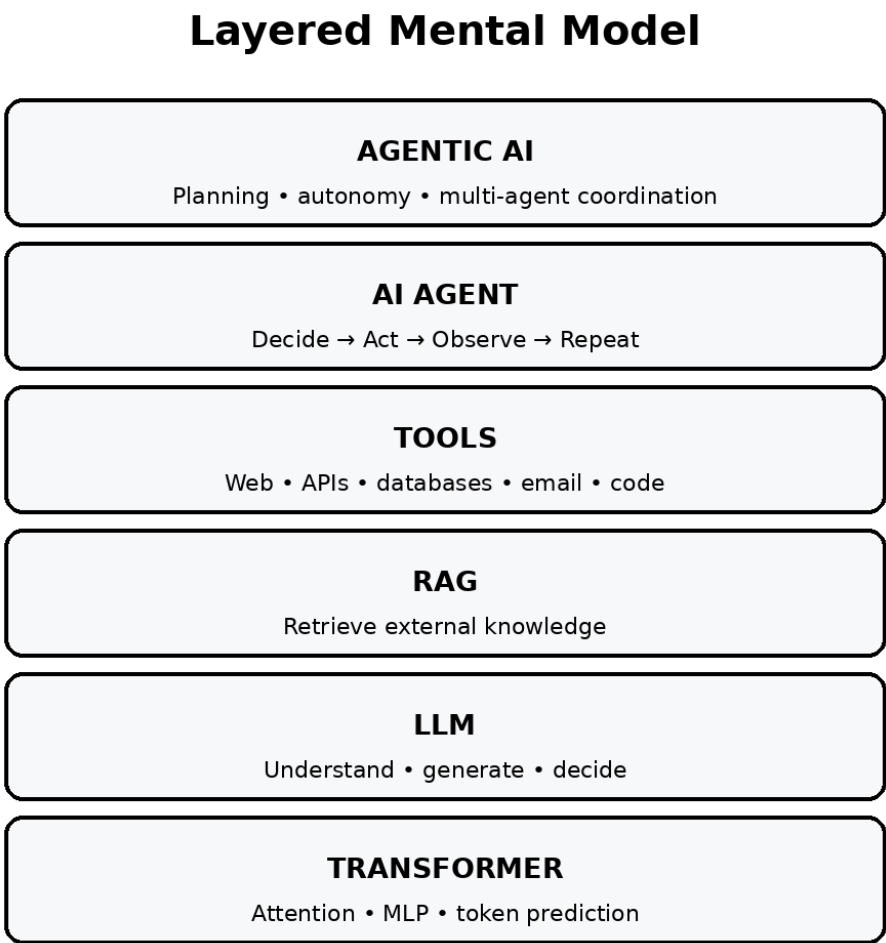
System	Input	Core behavior	Typical output
LLM	Prompt/context	Generate or transform information	Text or structured output
RAG	Question + retrieved context	Retrieve relevant knowledge, then generate	Grounded response

AI Agent	Goal + tools + state	Decide, act, observe and repeat	Completed task/result
----------	----------------------	---------------------------------	-----------------------

15. From AI Agent to Agentic AI

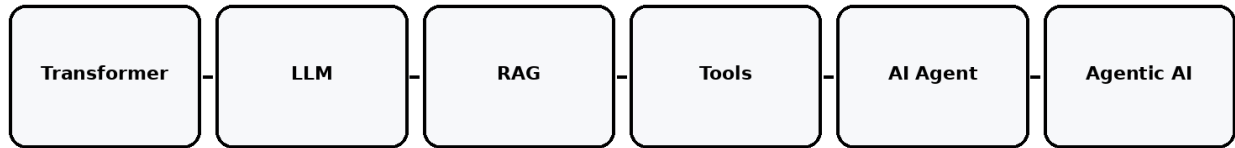
A bounded AI agent may complete one task through a small number of actions. Agentic AI describes systems with richer autonomy, planning, longer workflows, persistent state, dynamic tool use, or multiple cooperating specialized agents. Multi-agent systems may use supervisor, research, analysis, writing, coding, or validation agents.

16. Layered Mental Model



This provides a useful learning progression: Transformer → LLM → RAG → Tools → AI Agent → Agentic AI.

Progression of Modern LLM Systems



17. Key Formula

AI Agent = LLM + Instructions + Tools + State/Memory + Control Loop + Guardrails

Goal → Decide → Act → Observe → Update State → Decide Again → ... → Finish

18. Agent Frameworks

Frameworks such as LangGraph, CrewAI, PydanticAI, and similar agent-development systems provide abstractions for implementing parts of this architecture. Depending on the framework, these can include tool integration, workflow graphs, state management, structured outputs, retries, validation, tracing, and multi-agent coordination.